1. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она или можно внести некоторые доработки.
   Вы обратили внимание на таблицу с информацией о зарегистрировавшихся покупателях (клиентах).

Таблица имеет следующую схему:

clients (

client_id number, --уникальный id клиента

client_name varchar(255), --имя клиента

client_surname varchar(255), --фамилия клиента

login varchar(30), --логин, который придумал клиент

city_id number, --id города, который указал клиент (в интерфейсе выбирается название города, а в таблицу сохраняется id)

age number, --возраст клиента

reg_date date –дата регистрации на сайте

);

Считаете ли вы данный набор и смысл полей корректным? Если нет, то напишите что по вашему мнению некорректно и какие изменения внесли бы.

Ответ:

clients (

client_id int primary key, -- уникальный ID клиента

client_name varchar(20), --имя клиента

client_surnamevarchar(30), --фамилия клиента

email varchar(40) not null, -- email клиента

login varchar(32) not null, --логин, который придумал клиент

password varchar(130) not null, -- пароль

phone varchar(20) not null, -- номер телефона

gender_id int references gender on delete set null, -- ID пол клиента из таблицы gender

city_id int references cities on delete set null, -- ID города из таблицы cities

birthdate date not null, -- дата рождения клиента

reg_date date not null -- дата регистрации на сайте

);

gender (

gender_id serial primary key,

gender char unique not null

);

cities (

city_id int primary key, -- уникальный ID города

city_name varchar(30) -- название города

);

Добавлена возможность хранения email, пароля, телефона, пола и даты рождения клиента. Добавлено использование внешних ключей для связей с таблицами gender и cities. Удален возраст клиента, вместо этого теперь используется дата рождения. Изменены размеры переменной varchar().

---

2. Представьте, что вы устроились работать Дата-инженером в некоторый интернет магазин. До вас в этой компании уже работал один разработчик, который придумал небольшую базу данных для этого магазина и потом неожиданно уволился. Ваша задача провести проверку существующей архитектуры и решить корректна ли она или можно внести некоторые доработки.
   Вы обратили внимание на таблицу с информацией о товарах магазина.

Таблица имеет следующую схему:

```
items (
        item_id          number, --уникальный id товара
        item_name        varchar(255), --наименование товара
        item_cost        number –стоимость товара
);
```

Глядя на эту таблицу вы вспоминаете, что мы живём в рыночной экономике и цена, если не каждый день, то каждый месяц может меняться. К примеру, сезонные повышения цен или наоборот скидки (перед началом учебного года или новым годом). Так же вы знаете что у магазина есть отдел финансовой отчётности, в котором сотрудникам нужно строить различны отчёты (о доходах например) за разные промежутки времени, в том числе и за прошлые месяцы и даже прошлые года.

Как бы вы доработали архитектуру таблицы, что бы обеспечить историческое хранение стоимости товара?

Ответ:
Можно добавить столбец item_cost_date в таблицу items, который будет хранить дату, когда была установлена текущая цена.

```
items (
   item_id int primary key, --уникальный id товара
   item_name varchar(255) not null, --наименование товара
   item_cost numeric not null check(item_cost >0), --стоимость товара
   item_cost_date date not null -- дата установки текущей цены
);
```

---

3. Требуется проверить запрос на корректность и исправить там ошибки, если они есть. Существует учебная схема HR, содержащая таблицы: employees, departments и locations Необходимо получить все отделы0, расположенные в Сеуле в которых все сотрудники, не имеющие менеджера, зарабатывают в общей сложности более 100000.
   Для решения задачи был написан следующий запрос:

```
SELECT DEPARTMENT_ID,
       DEPARTMENT_NAME,
       SUM(SALARY) TOTAL_SALARY
  FROM EMPLOYEE E,
       DEPARTMENTS D
 WHERE E.DEPARTMENT_ID = D.DEPARTMENT_ID
   AND MANAGER_ID = NULL
   AND LOCATION_ID = (SELECT LOCATION_ID
                      FROM LOCATIONS
                     WHERE CITY = 'SEOUL')
   AND SUM(SALARY) >= 100000
 GROUP BY DEPARTMENT_NAME
```

Ответ:

```
SELECT d.DEPARTMENT_ID, d.DEPARTMENT_NAME
FROM DEPARTMENTS D
JOIN LOCATIONS l ON d.LOCATION_ID = l.LOCATION_ID
JOIN EMPLOYEES E ON d.DEPARTMENT_ID = e.DEPARTMENT_ID
WHERE l.CITY = 'SEOUL' AND e.MANAGER_ID IS NULL
GROUP BY d.DEPARTMENT_ID, d.DEPARTMENT_NAME
HAVING SUM(e.SALARY) > 100000;
```

---

4. В некоторой комнате на пол уронили карандаш. Объясните почему вы не можете через него перепрыгнуть?

Ответ:
Карандаш упал у стены.